

Simplicity Goes Far: Auditing Prompt Optimizers with Demonstration-Conditioned Prompt Search

Bohe Chen^{1,2}

¹ Beijing University of Posts and Telecommunications, Beijing, China

² Queen Mary University of London, London, UK

bhchen@bupt.edu.cn

Abstract. Automatic prompt optimization is growing more elaborate. Recent methods stack policy-gradient updates, trace reflection, and evolutionary search. These methods share a generate-and-select step; without controlling for it, reported gains may conflate the contributions of optimizer mechanisms and larger sampling budgets. We isolate this step as **Demonstration-Conditioned Prompt Search (DCPS)**, repurposing the Automatic Prompt Engineer (APE) recipe as a mechanism-attribution control: it freezes the generator, conditions it on randomly sampled demonstrations, scores candidates on validation data, and keeps the best—no learned update, reflection, or evolution. Across two optimizer families, DCPS replaces Proximal Policy Optimization (PPO) in StablePrompt, matching classification accuracy within 0.5 pp at roughly one-third the measured cost; in DSPy-based compound AI systems, under the reported budgets, it matches GEPA’s macro accuracy on GPT-4.1-mini while using 55% fewer rollouts at 32% lower measured API cost on average. The audit reveals a task-dependent pattern: proposal and selection recover much of heavier optimizers’ performance on structured and instruction-following tasks, while additional mechanisms retain advantages in some open-ended and reasoning-heavy settings. We recommend benchmarking prompt optimizers against generate-and-select controls and reporting rollout counts, token usage, and dollar costs alongside accuracy. Code and artifacts: <https://github.com/bhc6/DCPS>.

Keywords: Prompt Optimization · Mechanism Audit · Prompt Search · Cost Analysis

1 Introduction

Automatic prompt optimization (APO) has become increasingly elaborate. Reinforcement learning (RL) methods [3,12] often rely on long rollout streams and policy-optimization pipelines, while reflection-based methods [1] use natural-language feedback to diagnose failures and rewrite prompts. These mechanisms can improve peak accuracy, but they also increase compute and API costs and make attribution harder. When a prompt optimizer works, is the gain coming

from policy updates, reflection, and Pareto-style evolution, or from a simpler process of proposing prompts and selecting the best one on validation data?

To answer this question, we study **DCPS (Demonstration-Conditioned Prompt Search)**, a deliberately minimal control built from the Automatic Prompt Engineer (APE) generate-and-select pattern [27]. Unlike APE, which introduced this pattern as a standalone prompt-search algorithm, DCPS repurposes it as a control for mechanism attribution: the same generate-and-select loop is inserted into existing optimizer settings to probe what policy updates, reflection, merging, or Bayesian search add beyond validation-based selection. Demonstrations condition the generator to propose prompts but are not automatically carried into the final target-model prompt; DCPS is therefore neither a few-shot example selector nor a new optimizer, but a diagnostic tool whose contribution lies in the audit conclusions it enables.

We apply this control in two representative APO settings, differing only in the selection rule. **StablePrompt-DCPS** instantiates DCPS inside StablePrompt [12], freezing the generator and removing Proximal Policy Optimization (PPO) updates while preserving demonstration-conditioned proposal and Top- K selection. **DCPS-Compound** instantiates the same control on GEPA-style compound AI systems [1], keeping the task and rollout protocol but removing trace reflection, prompt mutation, merging, and Pareto evolution. We further report the Relative Cost-Efficiency Index (RCEI) as an auxiliary view of the token–performance trade-off.

Contributions.

1. **A reusable audit methodology.** We formalize DCPS as a four-stage control template and instantiate it inside two structurally different optimizer families—an RL-based pipeline (StablePrompt) and compound AI systems (GEPA and the heavy-budget MIPROv2 configuration)—to probe how much individual optimizer mechanisms add beyond validation-based selection.
2. **A diagnostic boundary for optimizer complexity.** Across both settings, proposal and selection recover most gains on structured tasks at lower cost, while policy updates and reflection retain advantages on some open-ended and reasoning-heavy tasks. The observed pattern identifies where optimizer overhead may be justified without treating task category as a deterministic boundary.
3. **Cost-aware evaluation.** We report accuracy jointly with rollout counts, token usage, and dollar costs, and use RCEI as an auxiliary cost-efficiency view, giving a replicable protocol for comparing optimizers under their actual budgets.

2 Related Work

Generate-and-select controls. Instruction Induction [7] and APE [27] elicit natural-language instructions from demonstrations and select them by task performance. Notably, APE itself reports that its optional iterative (Monte Carlo)

search saturates within a few rounds and is disabled by default—an early hint that added search contributes little once proposal and selection are in place. DCPS inherits this structure but changes its role: rather than a standalone optimizer, it is a controlled ablation for existing pipelines, systematizing that hint across policy updates, reflection, and evolution.

RL updates and reflection loops. Existing APO methods [19] split into two paradigms. RL-based optimizers train an auxiliary generator to reshape discrete token distributions (RLPrompt, StablePrompt, PRewrite, MORL-Prompt [3,12,11,8]). StablePrompt, for instance, adopts an adaptive PPO variant because vanilla PPO collapses on discrete prompt tokens. Reflection-based methods (ProTeGi [16], TextGrad [26], PromptAgent [22], GEPA [1]) instead use an LLM to read execution traces and propose edits, often with UCB, beam/tree search, or Pareto selection.

Compound AI systems and cost controls. Compound systems chain language modules under shared control flow. DSPy [10] treats the program as a compiler target for joint prompt optimization; MIPROv2 [15] adds Bayesian prompt search and GEPA [1] reflective rewriting with Pareto merging, often at thousands of rollouts. In parallel, RLHF/RLVR analyses suggest some gains credited to policy updates are reachable by sampling and selection: Yue et al. [25] use $\text{pass}@k$ to argue RLVR mainly sharpens the base model’s sampling distribution, and RAFT [4] shows best-of- N reward-ranked fine-tuning approaches PPO-style alignment. DCPS brings the same control to prompt optimization, with RCEI making marginal returns per token explicit.

3 Demonstration-Conditioned Prompt Search

3.1 Task Definition

Following the definition of compound AI systems [10,1], we write the target system as $\Phi = (M, C, \mathcal{X}, \mathcal{Y})$, where $M = \langle M_1, \dots, M_{|M|} \rangle$ is a sequence of language modules, C is the control flow, and $\mathcal{X}, \mathcal{Y}$ are the global input and output schemas. Each module $M_i = (\pi_i, \theta_i, \mathcal{X}_i, \mathcal{Y}_i)$ contains a prompt π_i , fixed parameters θ_i , and local schemas. Let Π_Φ collect all module prompts and Θ_Φ the fixed target-model weights. Given a task distribution $\mathcal{T}$ over instances (x, m) and a metric $\mu \in [0, 1]$, prompt optimization seeks, under rollout budget B :

$$\Pi_\Phi^* = \arg \max_{\Pi_\Phi} \mathbb{E}_{(x,m) \sim \mathcal{T}} [\mu(\Phi(x; \Pi_\Phi, \Theta_\Phi), m)] \quad (1)$$

A rollout is one evaluation call to Φ under a candidate prompt configuration. This covers both single-prompt benchmarks ($|M| = 1$) and compound pipelines (where Π_Φ is jointly optimized across modules).

3.2 DCPS Pipeline

DCPS uses one optimization signal: the validation performance of prompts generated from randomly sampled demonstrations. The demonstrations shape the

generator’s proposal but do not by themselves become part of the final inference prompt. Given a target system Φ , a training set D_{train} , a fixed validation subset $D_{\text{val}}^{\text{sub}}$, a frozen generator $p_{\theta_G^{(0)}}$, and metric μ , DCPS iterates four stages on the candidate prompt set:

- **Demonstration sampling $\mathcal{S}$.** Sample k examples at random from D_{train} and format them into a meta-prompt $\mathcal{I}^{(t)}$ for the generator.
- **Candidate generation $\mathcal{G}_{\theta_G^{(0)}}$.** Draw candidate prompts $\Pi^{(t)} \sim p_{\theta_G^{(0)}}(\cdot \mid \mathcal{I}^{(t)})$ from the frozen generator, with no parameter update, trace-based reflection, or rewrite loop.
- **Evaluation $\mathcal{E}$.** Score each generated prompt on the fixed validation subset $D_{\text{val}}^{\text{sub}}$ under μ .
- **Selection $\mathcal{F}$.** Retain the best candidate(s): greedy replacement in compound runs, or Top- K retention in StablePrompt-DCPS.

The resulting control operator is

$$\mathcal{O}_{\text{DCPS}}(\mathcal{T}, \Phi, B) = \mathcal{F} \circ \mathcal{E} \circ \mathcal{G}_{\theta_G^{(0)}} \circ \mathcal{S}. \quad (2)$$

At evaluation time the target system receives the generated prompt $\Pi^{(t)}$; the sampled demonstrations affect it only through the generator’s meta-prompt, unless the generator copies them into its output. This separates DCPS from few-shot example selection and from few-shot in-context evaluation, while making the borrowed component explicit: APE-style demonstration-conditioned proposal coupled to score-based selection [27], without policy updates, trace reflection, prompt mutation, or merge operations.

3.3 Two DCPS Audits

We instantiate the DCPS control template in two settings, comparing against StablePrompt, GEPA, and MIPROv2-Heavy (the heavy-budget MIPROv2 configuration) as three representative sources of complexity in modern APO: policy-gradient updates, trace reflection with prompt evolution, and Bayesian search over instruction–demonstration programs. The selector is matched to each host: StablePrompt-DCPS keeps StablePrompt’s Top- K queue and training-batch rewards, while DCPS-Compound uses greedy validation replacement. These serve as a *policy-update audit* of StablePrompt and a *compound-system audit* against GEPA-style reflection and MIPROv2-style Bayesian search (Algorithms 1 and 2).

StablePrompt-DCPS audit. StablePrompt [12] already contains the proposal stage DCPS isolates: an agent generator receives randomly sampled examples in a meta-prompt, proposes candidates, and is then updated with PPO-style rewards. **StablePrompt-DCPS** (Algorithm 1) freezes the initial generator weights $\theta_G^{(0)}$ and removes PPO updates, while keeping random k -shot sampling, the original reward computation, and Top- K retention. It also inherits

Algorithm 1 StablePrompt-DCPS

Require: Training set D_{train} , validation set $D_{\text{val}}^{\text{demo}}$, frozen generator $p_{\theta_G^{(0)}}$, target system Φ , metric μ , epochs T , candidates per batch N , few-shot size k , queue capacity K

- 1: $\mathcal{P} \leftarrow \emptyset$, $t \leftarrow 0$
- 2: **for** $e = 1$ to T **do**
- 3: **for** $B_j \in \text{Batches}(D_{\text{train}})$ **do**
- 4: $t \leftarrow t + 1$
- 5: $E^{(t)} \leftarrow \text{Sample}(D_{\text{val}}^{\text{demo}}, k) \{S\}$
- 6: $\mathcal{I}^{(t)} \leftarrow \text{FmtMeta}(E^{(t)})$
- 7: $\{\Pi_i^{(t)}\}_{i=1}^N \sim p_{\theta_G^{(0)}}(\cdot \mid \mathcal{I}^{(t)}) \{\mathcal{G}\}$
- 8: **for** $i = 1$ to N **do**
- 9: $s_i^{(t)} \leftarrow \text{Eval}(\Pi_i^{(t)}, B_j) \{\mathcal{E}\}$
- 10: $c_i^{(t)} \leftarrow (\Pi_i^{(t)}, s_i^{(t)})$
- 11: $\mathcal{P} \leftarrow \text{TopK}(\mathcal{P} \cup \{c_i^{(t)}\}, K) \{\mathcal{F}\}$
- 12: **end for**
- 13: **end for**
- 14: **end for**
- 15: **return** $\mathcal{P}$

Algorithm 2 DCPS-Compound

Require: Training set D_{train} , validation set D_{val} , frozen generator $p_{\theta_G^{(0)}}$, target system Φ , metric μ , iterations I , few-shot size k , val. subset n_v

- 1: $\Pi^* \leftarrow \text{null}$, $s^* \leftarrow -\infty$; $D_{\text{val}}^{\text{sub}} \leftarrow \text{Sample}(D_{\text{val}}, n_v)$
- 2: **for** $t = 1$ to I **do**
- 3: $E^{(t)} \leftarrow \text{Sample}(D_{\text{train}}, k) \{S\}$
- 4: $\mathcal{I}^{(t)} \leftarrow \text{FmtMeta}(E^{(t)})$
- 5: $\Pi^{(t)} \sim p_{\theta_G^{(0)}}(\cdot \mid \mathcal{I}^{(t)}) \{\mathcal{G}\}$
- 6: $s^{(t)} \leftarrow \text{Eval}(\Pi^{(t)}, D_{\text{val}}^{\text{sub}}) \{\mathcal{E}\}$
- 7: **if** $s^{(t)} > s^*$ **then**
- 8: $\Pi^* \leftarrow \Pi^{(t)}$; $s^* \leftarrow s^{(t)} \{\mathcal{F}\}$
- 9: **end if**
- 10: **end for**
- 11: **return** Π^*

Fig. 1. Left: StablePrompt-DCPS. Right: DCPS-Compound. $\text{FmtMeta}(E)$ formats the k -shot examples into a meta-prompt for the generator; $\text{Eval}(\Pi, D) = \frac{1}{|D|} \sum_{(x,m) \in D} \mu(\Phi(x; \Pi, \Theta_\Phi), m)$. In StablePrompt-DCPS, $D_{\text{val}}^{\text{demo}}$ supplies only demonstrations and rewards are computed on training batches B_j .

StablePrompt’s data protocol: demonstrations are drawn from $D_{\text{val}}^{\text{demo}}$ and candidates are scored on training batches, reversing the default DCPS data roles in §3.2. Applied identically to PPO and DCPS, this protocol leaves the comparison unaffected. The gap between StablePrompt-PPO and StablePrompt-DCPS estimates what the policy update adds beyond proposal and selection.

DCPS-Compound for compound AI systems. For compound AI systems, DCPS-Compound follows the task and rollout protocol used for GEPA-family and MIPROv2-Heavy optimizers [1,15] (Algorithm 2), auditing what trace reflection, Pareto selection, and Bayesian instruction–demonstration search add beyond the generate-and-select control. Each iteration samples demonstrations from D_{train} , queries the frozen generator, evaluates on $D_{\text{val}}^{\text{sub}}$, and replaces the incumbent when validation improves.

4 Experimental Setup

We evaluate StablePrompt-DCPS on the original StablePrompt benchmarks and DCPS-Compound against GEPA, GEPA-MERGE, and MIPROv2-Heavy

on compound systems. The runs reuse the original implementations and published defaults; costs are measured from WandB summaries, local logs, and API-provider records.

4.1 StablePrompt-DCPS Ablation

We follow Kwon et al.’s full StablePrompt configuration [12]. Both the target and agent models are **gemma-1.1-7B-it** [5]. We evaluate this setup on the same three task families with the original scoring protocols: (1) **text classification** (6 GLUE/SuperGLUE subsets [21,20]), (2) **multi-task QA** (57-subject MMLU [6]), and (3) **instruction induction & generation** (24 Instruction Induction (II) [7] + 18 Big-Bench Instruction Induction (BBII) [27] subsets, split into BBII-TC and BBII-Gen). Removing gradient computation and PPO optimizer states lets DCPS run inference-only on one NVIDIA A40 48 GB, whereas StablePrompt-PPO follows the original A100 80 GB setup. In our reproduction, we fix a train/test overlap in II and missing batch weights in StablePrompt’s Softmax-difference metric. Following StablePrompt, we report the mean over three seeds of the best test score among the Top-5 prompts ranked by training reward; this protocol is optimistic in absolute terms but is applied identically to PPO and DCPS, so the PPO–DCPS comparison is unaffected.

4.2 Compound AI System Optimization Setup

We evaluate two backbones, **Qwen3-8B** [18] and **GPT-4.1-mini** [14], using each as both the target system and the prompt generator. The six benchmarks cover multi-step retrieval tasks, HotpotQA [24] and HoVer [9], plus single- or two-stage reasoning and control tasks: IFBench (verifiable instruction following) [17], PUPA (PAPILLON privacy delegation) [13], AIME-2025 [2], and LiveBench-Math [23]. DCPS-Compound uses task-specific rollout budgets and validation subset sizes $n_v \in [15, 50]$; for each replicate, the reported prompt ranks first by validation score, and when multiple logged replicates are available, we select the displayed result using validation performance only, never test scores.

4.3 Cost and Efficiency Metrics

We report accuracy together with rollout budgets and measured cost. A *rollout* is one target-system evaluation or generator call, so the per-task DCPS-Compound budget has the closed form rollouts = $(n_v + 1) \times I$, where n_v is the validation-subset size and I the number of iterations. This reproduces every rollout budget in Table 2(b) (AIME-2025 320, IFBench/LiveBench-Math 620, PUPA 920, HotpotQA 1,020, HoVer 7,068; macro 1,761). Validation dominates the count (e.g. 50 of every 51 HotpotQA rollouts), so DCPS’s cost is due almost entirely to proposal and validation. For cost-normalized comparison, we use relative error reduction (RER), which normalizes gains by the remaining headroom over the unoptimized prompt:

$$\text{RER}(\mathcal{M} \parallel \mathcal{B}) = \frac{\mathcal{M} - \mathcal{B}}{100\% - \mathcal{B}} \times 100\%, \quad (3)$$

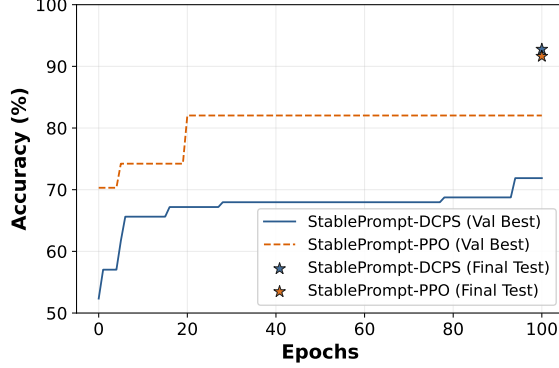


Fig. 2. Running-best SST-2 validation accuracy for StablePrompt-DCPS vs. PPO on `gemma-1.1-7B-it`; stars mark the final test score at epoch 100.

where $\mathcal{B}$ is the unoptimized prompt score and $\mathcal{M}$ is the score achieved by the optimized method. Cost is reported in millions of price-weighted tokens across optimization and final test evaluation:

$$T = T_{\text{optimizer}} + T_{\text{test}}, \quad T_{(\cdot)} = T_{\text{in}}^{(\cdot)} + 4T_{\text{out}}^{(\cdot)}, \quad (4)$$

using the approximate GPT-4.1-mini output-to-input price ratio. RCEI reports relative error reduction per log cost multiplier:

$$\text{RCEI}(\mathcal{M} \parallel \mathcal{B}) = \frac{\text{RER}(\mathcal{M} \parallel \mathcal{B})}{\log_2(T_{\mathcal{M}}/T_{\mathcal{B}})}. \quad (5)$$

Since optimization always consumes tokens, $T_{\mathcal{M}} > T_{\mathcal{B}}$ and the denominator is strictly positive. RCEI is an auxiliary view, reported with accuracy, rollout counts, and dollar costs; macro RER and RCEI are arithmetic means over per-task values.

5 Main Results and Diagnostic Findings

We report accuracy alongside rollout counts and costs for both audits. §5.1 uses StablePrompt-DCPS to ask what PPO adds over the frozen-generator control, and §5.2 asks the same of reflection and Bayesian search. The two differ in granularity: the StablePrompt study is a single-variable ablation, while the compound-system study is a control-based cost–performance comparison, not a budget-matched ranking.

5.1 StablePrompt-PPO vs StablePrompt-DCPS

Table 1 shows PPO updates have limited marginal value on classification: on GLUE/SuperGLUE [21,20] and BBII-TC [27], DCPS and PPO do not differ

Table 1. StablePrompt-PPO vs. StablePrompt-DCPS on `gemma-1.1-7B-it`. Scores are macro-averages over subsets; Δ (PPO–DCPS), 95% CI, and Cohen’s d are computed from per-subset differences (vs. PPO@30 epochs for extended II runs). Times and measured GPU costs are reported per run.

Task Family	PPO	DCPS	Δ	95% CI	d	DCPS (h/\$)	PPO (h/\$)	Note
GLUE/SuperGLUE	76.7	77.1	−0.4	[−2.30, +1.97]	−0.14	4.83/1.93	3.97/5.96	comparable
BBII-TC	57.6	57.2	+0.4	[−0.68, +1.63]	+0.20	3.38/1.35	3.17/4.76	comparable
BBII-Gen	63.0	60.7	+2.3	[+0.15, +4.35]	+0.79	2.66/1.06	1.84/2.76	PPO better
II @30 epochs	48.7	44.3	+4.4	[+0.15, +8.70]	+0.41	2.76/1.10	6.65/9.98	PPO better
II @100 epochs	—	48.6	+0.1	[−2.67, +2.63]	+0.01	14.74/5.90	—	gap closed
MMLU	55.9	54.1	+1.8	[+0.99, +2.74]	+0.53	6.58/2.63	6.38/9.57	PPO better

significantly ($|\Delta| \leq 0.4$ pp), and Figure 2 shows DCPS tracking PPO throughout SST-2 training. The picture changes where behavior is less template-like: PPO leads by +2.3 pp on BBII-Gen [27] and +1.8 pp on MMLU [6], so policy updates still help when the target is hard to reach by random conditioning. On II [7], extending DCPS to 100 epochs closes the gap with PPO at 30 epochs (+0.1 pp). Because PPO is not re-run at matching epochs, this result suggests that extra search budget can substitute for the policy update; it does not show that DCPS strictly dominates. Where accuracy is matched, removing the policy update eliminates gradient computation and optimizer state, enabling inference-only runs on a cheaper GPU tier: measured single-run cost falls from \$4.76–\$5.96 (A100) to \$1.35–\$1.93 (A40), about one-third as much under our rates.

5.2 DCPS on Compound AI Systems

Table 2(a) summarizes the cost–performance trade-off. With less than half of GEPA’s average rollout budget (1,761 vs. 3,936) and a lower API cost, DCPS-Compound attains both the highest macro RER (+26.23%) and the highest macro RCEI among optimized methods, while matching GEPA’s absolute macro accuracy on GPT-4.1-mini (61.14, single seed; see §6.2). MIPROv2-Heavy [15], run at GEPA’s full budget, trails on both relative error reduction and cost-efficiency.

The main table reports one selected rollout budget per method–task cell: GEPA-family methods at their published defaults and DCPS-Compound at the budgets in Table 2. For HoVer, a small-budget DCPS run matching the other tasks was weak, so the table reports the GEPA-aligned extended run. §5.3 reports a budget-sensitivity analysis on three tasks, leaving a full sweep for future work.

Table 2(b) reveals the underlying mechanism boundary. DCPS-Compound is strongest on single- or two-stage tasks that hinge on precise instruction following (IFBench, PUPA), and on HoVer once adequately budgeted (§5.3). GEPA’s advantages are concentrated on reasoning-heavy tasks: HotpotQA (+6.00 pp on GPT-4.1-mini), LiveBench-Math (both backbones), and AIME-2025 with Qwen3-8B. DCPS offsets these on IFBench, PUPA, and budgeted HoVer, so the GPT-4.1-mini macro tie reflects offsetting per-task results, not method equivalence. GEPA-MERGE, by contrast, falls below even the unoptimized Qwen3-8B

Table 2. Compound-system results. **(a)** Cost–performance summary on GPT-4.1-mini, macro-averaged across the six benchmarks ($\log_2(T_{\mathcal{M}}/T_{\mathcal{B}})$ with T as in §4.3). **(b)** Task-level accuracy on Qwen3-8B and GPT-4.1-mini; macro-average is the unweighted mean over six benchmarks, and the bottom block lists per-task rollout budgets (shared across backbones). In (b), LB-Math = LiveBench-Math and AIME = AIME-2025.

(a) Cost–performance summary (GPT-4.1-mini, macro-averaged)								
Method ($\mathcal{M}$)	RER	$T_{\mathcal{M}}$	$\overline{\log_2(T_{\mathcal{M}}/T_{\mathcal{B}})}$	RCEI	Rollouts	Cost		
Baseline (unoptimized prompt, $\mathcal{B}$)	0	1.06	0	–	–	\$0.43		
MIPROv2-Heavy	+12.70%	27.11	4.67	+2.66	3,936	\$10.84		
GEPA	+24.15%	28.36	4.73	+5.30	3,936	\$11.34		
GEPA-MERGE	+24.87%	24.07	4.50	+5.90	3,936	\$9.63		
DCPS-Compound	+26.23%	19.26	3.85	+7.20	1,761	\$7.70		
(b) Task-level accuracy								
Backbone	Method	Hotpot	IFBench	HoVer	PUPA	AIME	LB-Math	Macro
Qwen3-8B	Baseline	40.33	38.61	33.67	81.55	47.33	65.01	51.08
	MIPROv2-Heavy	58.67	38.61	44.00	78.67	58.00	67.46	57.57
	GEPA	63.33	40.65	53.33	87.57	59.33	70.57	62.46
	GEPA-MERGE	61.33	29.08	52.00	84.81	62.67	66.28	59.36
	DCPS-Compound	61.33	43.88	56.67	90.02	55.33	65.08	62.05
GPT-4.1-mini	Baseline	36.10	48.13	40.33	80.81	40.00	55.80	50.20
	MIPROv2-Heavy	55.00	51.19	47.33	83.59	46.67	57.30	56.85
	GEPA	65.00	49.83	49.67	90.10	48.00	64.21	61.14
	GEPA-MERGE	63.33	49.49	49.00	93.52	46.67	61.14	60.53
	DCPS-Compound	59.00	51.53	54.67	94.09	48.00	59.52	61.14
<i>Total optimization budget (# rollouts)</i>								
GEPA/GEPA-MERGE/MIPROv2-Heavy		6,871	3,593	7,051	2,426	1,839	1,839	3,936
DCPS-Compound		1,020	620	7,068	920	320	620	1,761

IFBench baseline, indicating that crossover-based prompt merging can be unstable on strict-format tasks.

Prompt length and the shapes of optimized prompts. Beyond rollouts and dollars, the optimized prompts differ sharply in length (Table 3). DCPS produces the shortest optimized prompt on every benchmark, measured in both characters and tokens counted with `tiktoken`’s `cl100k_base` encoding. The ratios are not interchangeable: because DCPS’s LiveBench-Math prompt is LaTeX-dense, it is $3.4\times$ shorter than GEPA’s in characters but $2.9\times$ shorter in tokens; we therefore base cost claims on token counts. The gap is qualitative: DCPS specifies a compact *output contract* while GEPA accumulates a long domain *playbook*. On PUPA (GPT-4.1-mini), DCPS’s 742-character redaction-module prompt states one transferable redaction rule and beats the baseline by +13.3 pp, whereas GEPA’s 6,074-character prompt hard-codes trace-distilled carve-outs that do not transfer to the held-out non-programming mix and scores below DCPS (90.10 vs. 94.09). Where the reward is a transferable contract, the minimal prompt can generalize well; where GEPA leads, as on HotpotQA and LiveBench-Math, its longer trace-conditioned prompts are consistent with—but do not by themselves

Table 3. Optimized-prompt length in characters / tokens (GPT-4.1-mini), with tokens counted using tiktoken’s `cl100k_base` encoding. DCPS is shortest on every benchmark.

Benchmark	DCPS (char / tok)	GEPA	MIPROv2-Heavy
AIME-2025	1,339 / 227	6,625 / 1,330	4,460 / 748
LiveBench-Math	1,536 / 337	5,295 / 984	5,519 / 950
HoVer	2,471 / 460	12,135 / 2,309	16,401 / 2,922
HotpotQA	3,009 / 591	10,479 / 2,105	14,681 / 2,521
IFBench	1,458 / 259	6,598 / 1,254	11,027 / 1,893
PUPA	1,459 / 270	6,074 / 1,180	11,021 / 1,978

establish—a benefit from localized feedback. Verbatim prompts for all cells are in the released artifacts.

5.3 Budget Sensitivity and the Generalization Gap

The per-task budgets in Table 2 raise a natural question: when does a larger roll-out budget help, and when does it merely pressure the selector? We extend DCPS on three diagnostic tasks under the same validation-only selection rule, increasing the HotpotQA budget to 6,120 rollouts, the HoVer budget toward GEPA scale, and the LiveBench-Math budget to 1,860 rollouts. These runs are not new leaderboard entries; they test whether validation-selected prompts continue to generalize as the candidate pool grows. For HoVer, the main table reports the extended DCPS run because the initial small-budget run (620 rollouts, matching the other small-budget tasks) was clearly under-budgeted (44.00 on Qwen3-8B and 45.67 on GPT-4.1-mini).

The probes separate three regimes. **HoVer** is mainly a coverage bottleneck: with GEPA-scale sampling, DCPS surpasses GEPA on both backbones while keeping validation–test gaps small (≤ 3.3 pp), so the initial GEPA advantage was a budget artifact. **HotpotQA** is different: extending DCPS to 6,120 rollouts raises GPT-4.1-mini from 59.00 to 63.67 (narrowing the gap to GEPA’s 65.00 from 6.00 to 1.33 pp) but leaves Qwen3-8B essentially flat (61.33 $\rightarrow$ 60.33, vs. GEPA’s 63.33). Extra rollouts thus narrow the gap on GPT-4.1-mini but do not remove GEPA’s edge on either backbone. This pattern is consistent with trace-level feedback helping to localize failures across retrieval and reasoning steps, but the present comparison does not isolate trace feedback from GEPA’s other search mechanisms.

LiveBench-Math is the cautionary case: more rollouts raise validation accuracy but lower held-out accuracy (gaps of 20.95 and 13.50 pp in single-seed results), so the rollout budget represents not only compute but also statistical pressure on the selector.

Failure modes: validation representativeness and demonstration sampling. Two factors govern when validation-only selection misleads (Table 4). First, *sub-*

set representativeness matters more than size: on AIME-2025 with Qwen3-8B (AIME-Qwen; 45 validation items), a fixed head slice of 15 inflates validation accuracy to 73.3 but transfers to only 46.0 on the test set, ≈ 9 pp worse than a seeded random sample of 15 ($66.7 \rightarrow 55.3$) or the full set ($55.6 \rightarrow 53.3$); we use random sampling throughout. Second, the val $\rightarrow$ test gap widens when the subset is small and validation accuracy saturates (IFBench, ≥ 44 pp) and shrinks as the subset grows (AIME-Qwen, 11.4 pp)—a winner’s-curse effect shared by any validation-selection method, including GEPA. Demonstration sampling (the $\mathcal{S}$ step) is a third lever: redrawing demonstrations introduces additional variance that the reported single-run points do not isolate; controlled noise injection and multi-seed analysis remain future work.

Table 4. Validation $\rightarrow$ test gap under validation-only selection. The gap is largest where the validation subset is small and validation accuracy saturates, and shrinks as the subset grows.

Task (backbone)	opt rollouts	val	test	gap
IFBench (GPT-4.1-mini)	620	96.7	51.5	45.2
IFBench (Qwen3-8B)	620	88.3	43.9	44.4
AIME-2025 (GPT-4.1-mini)	320	80.0	48.0	32.0
AIME-2025 (Qwen3-8B)	320	66.7	55.3	11.4

6 Discussion

6.1 Applicability Boundaries from Optimization Signal to Pipeline Complexity

The results show a task- and backbone-dependent pattern rather than a strict dichotomy. On tasks bottlenecked by output formatting or short-context instruction following, random demonstrations often give the frozen generator enough signal for proposal and selection to approach the heavier optimizers at lower cost. PPO and GEPA still retain advantages on some open-ended and reasoning-heavy cells, though the compound comparison does not isolate which of GEPA’s reflection and search components produce those gains. The control likely does well in the first group because a strong instruction-tuned generator already carries prompt-writing priors; consistent with this hypothesis, heavier optimizers lead by larger margins on some Qwen3-8B cells and reasoning-heavy tasks. This remains a hypothesis, not a controlled test of generator strength. The large validation–test gaps on IFBench and AIME-2025 further show that the apparent strength of any validation-selected optimizer depends on subset representativeness and candidate-pool size. Heavier optimizers should therefore be evaluated against a simple control using held-out performance, not validation performance alone.

6.2 Limitations and Empirical Boundaries

Non-identical settings: StablePrompt uses `gemma-1.1-7B-it`, whereas compound runs use Qwen3-8B and GPT-4.1-mini to match GEPA; the audits therefore examine each mechanism in its native setting. **Budget and replication:** the compound table reports one budget per method–task cell, and DCPS probes only three tasks. These validation-selected single runs establish cost–performance points, not a statistically resolved ranking. **Optimization capacity:** without trace rewriting or module-level attribution, DCPS cannot localize cascading errors; whether this explains its deficits requires component ablations. **Cost and coverage:** RCEI depends on token pricing and rental rates; code generation, summarization, tool use, and longer interactive tasks remain untested.

7 Conclusion

We asked how much performance attributed to complex prompt optimizers remains after reducing them to generation and validation-based selection. Under the evaluated settings, this control is cost-competitive on structured and instruction-following tasks, while PPO and GEPA retain advantages in some open-ended and reasoning-heavy settings. It should serve as a baseline, not replace heavier optimizers; evaluations should report validation–test gaps, rollout counts, token usage, dollar costs, and accuracy.

References

1. Agrawal, L.A., Tan, S., Soylu, D., Ziemis, N., Khare, R., Opsahl-Ong, K., Singhvi, A., Shandilya, H., Ryan, M.J., Jiang, M., Potts, C., Sen, K., Dimakis, A.G., Stoica, I., Klein, D., Zaharia, M., Khattab, O.: GEPA: Reflective prompt evolution can outperform reinforcement learning. In: ICLR (2026), oral
2. Balunovic, M., Dekoninck, J., Petrov, I., Jovanovic, N., Vechev, M.: MathArena: Evaluating LLMs on uncontaminated math competitions. <https://matharena.ai/> (2025)
3. Deng, M., Wang, J., Hsieh, C.P., Wang, Y., Guo, H., Shu, T., Song, M., Xing, E.P., Hu, Z.: RLPrompt: Optimizing discrete text prompts with reinforcement learning. In: EMNLP (2022)
4. Dong, H., Xiong, W., Goyal, D., Zhang, Y., Chow, W., Pan, R., Diao, S., Zhang, J., Shum, K., Zhang, T.: RAFT: Reward rAnked FineTuning for generative foundation model alignment. Transactions on Machine Learning Research (TMLR) (2023)
5. Gemma Team: Gemma: Open models based on Gemini research and technology. arXiv:2403.08295 (2024)
6. Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., Steinhardt, J.: Measuring massive multitask language understanding. In: ICLR (2021)
7. Honovich, O., Shaham, U., Bowman, S.R., Levy, O.: Instruction induction: From few examples to natural language task descriptions. In: ACL (2023)
8. Jafari, Y., Mekala, D., Yu, R., Berg-Kirkpatrick, T.: MORL-Prompt: An empirical analysis of multi-objective reinforcement learning for discrete prompt optimization. In: Findings of EMNLP (2024)

9. Jiang, Y., Bordia, S., Zhong, Z., Dognin, C., Singh, M., Bansal, M.: HoVer: A dataset for many-hop fact extraction and claim verification. In: Findings of EMNLP (2020)
10. Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Haq, S., et al.: DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In: ICLR (2024)
11. Kong, W., Hombaiah, S.A., Zhang, M., Mei, Q., Bendersky, M.: PRewrite: Prompt rewriting with reinforcement learning. In: ACL (2024)
12. Kwon, M., Kim, G., Kim, J., Lee, H., Kim, J.: StablePrompt: Automatic prompt tuning using reinforcement learning for large language model. In: EMNLP (2024)
13. Li, S., Raghuram, V.C., Khattab, O., Hirschberg, J., Yu, Z.: PAPILLON: Privacy preservation from internet-based and local language model ensembles. In: NAACL (2025)
14. OpenAI: GPT-4.1 series. <https://openai.com/index/gpt-4-1/> (2025)
15. Opsahl-Ong, K., Ryan, M.J., Purtell, J., Broman, D., Potts, C., Zaharia, M., Khattab, O.: Optimizing instructions and demonstrations for multi-stage language model programs. In: EMNLP (2024)
16. Pryzant, R., Iter, D., Li, J., Lee, Y., Zhu, C., Zeng, M.: Automatic prompt optimization with “Gradient Descent” and beam search. In: EMNLP (2023)
17. Pyatkin, V., Malik, S., Graf, V., Ivison, H., Huang, S., Dasigi, P., Lambert, N., Hajishirzi, H.: Generalizing verifiable instruction following. In: NeurIPS Datasets and Benchmarks (2025)
18. Qwen Team: Qwen3 technical report. arXiv:2505.09388 (2025)
19. Ramnath, K., Zhou, K., Guan, S., Mishra, S.S., Qi, X., Shen, Z., Wang, S., Woo, S., Jeoung, S., Wang, Y., et al.: A systematic survey of automatic prompt optimization techniques. In: EMNLP (2025)
20. Wang, A., Pruksachatkun, Y., Nangia, N., Singh, A., Michael, J., et al.: SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In: NeurIPS. vol. 32 (2019)
21. Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., Bowman, S.R.: GLUE: A multi-task benchmark and analysis platform for natural language understanding. In: ICLR (2019)
22. Wang, X., Li, C., Wang, Z., Bai, F., Luo, H., Zhang, J., Jojic, N., Xing, E.P., Hu, Z.: PromptAgent: Strategic planning with language models enables expert-level prompt optimization. In: ICLR (2024)
23. White, C., Dooley, S., Roberts, M., Pal, A., Feuer, B., Jain, S., Shwartz-Ziv, R., Jain, N., Saifullah, K., Dey, S., et al.: LiveBench: A challenging, contamination-limited LLM benchmark. In: ICLR (2025)
24. Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W.W., Salakhutdinov, R., Manning, C.D.: HotpotQA: A dataset for diverse, explainable multi-hop question answering. In: EMNLP (2018)
25. Yue, Y., Chen, Z., Lu, R., Zhao, A., Wang, Z., Yue, Y., Song, S., Huang, G.: Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In: NeurIPS (2025), oral
26. Yuksekgonul, M., Bianchi, F., Boen, J., Liu, S., Lu, P., Huang, Z., Guestrin, C., Zou, J.: TextGrad: Automatic “Differentiation” via text. *Nature* **639**, 609–616 (2025)
27. Zhou, Y., Muresanu, A.I., Han, Z., Paster, K., Pitis, S., Chan, H., Ba, J.: Large Language Models are Human-Level Prompt Engineers. In: ICLR (2023)